

CLASH OF THE **CLAWS**

RVCE_ATOMIC CLAWS

**Sentinel feels the device -
adapting every response to
your phone's heartbeat in
real time.**

Kavya Jain



Problem Statement

Why We Selected This Theme

On-Device AI + Daily Utility (Smartphones) - We chose this because it's the only theme where the problem physically exists on the device in your pocket right now. Every Samsung Galaxy user who runs Bixby, Samsung AI, or any on-device LLM has experienced this like the phone gets hot, the response stutters, the app crashes. It's not a theoretical problem. It happens daily. No one has fixed it because everyone treats the AI layer and the hardware layer as separate concerns. We connected them.

What Problem Are We Solving

On-device AI agents are **hardware-blind**. They execute inference tasks at full intensity regardless of whether the device has 4GB of RAM free or 200MB. Regardless of whether the SoC is at 45°C or 87°C. Regardless of whether the battery is at 90% or 8%.

The result is three specific failure modes:

- **The stutter** - thermal throttling kicks in mid-response because the SoC hit its limit. The AI output becomes fragmented or cuts off entirely.
- **The silent kill** - the Android OOM killer terminates the AI process when RAM pressure exceeds threshold. No warning. No graceful exit. The user just sees the app close.
- **The drain cliff** - sustained LLM inference pulls 4 - 6W of peak current. On a low battery this accelerates shutdown nonlinearly like the phone dies faster than the user expected.

All three happen *after* the OS acts. **Nobody is preventing them.**

Problem Statement

Who Faces It - The Personas

Persona 1 - Priya, 24, Engineering student, Hyderabad Uses Samsung Galaxy S23 for AI-assisted note-taking during 3 hour lectures. Phone regularly throttles in the final hour because the SoC has been running hot. AI responses become slow and incomplete exactly when she needs them most. Battery is at 15% and draining fast. She has no charger.

Persona 2 - Arjun, 34, Sales manager, Bangalore Uses Galaxy AI on his Galaxy S24 Ultra between client meetings. Running 12 apps in background. Asks the AI to summarise a 20-page contract while on a call. The AI process gets OOM-killed silently. He thinks the app crashed. He doesn't know it was a RAM kill.

Persona 3 - Samsung OEM / App Developer Building a productivity app that embeds an on-device LLM. Has no visibility into whether the AI is competing with the camera, GPS, or background sync for resources. Ships the app. Gets 1-star reviews saying "makes phone hot" and "drains battery." Has no telemetry to diagnose it.

Why Is It Important

It is the reason people distrust on-device AI. When AI on a phone feels unreliable — slow sometimes, fast other times, occasionally just dies — users stop using it. They go back to the cloud. Samsung's entire on-device AI strategy depends on users trusting that Galaxy AI works *consistently*, not just when the phone happens to be in a good state.

The numbers that make this concrete:

- A Galaxy S24 SoC throttles at **85°C** - sustained LLM inference can reach this in under **4 minutes** on a warm day
- OOM kills on Android happen at **~90% RAM pressure** with zero user warning
- Peak LLM inference draws **4–6W** - on a 5000mAh battery at 10%, that's under **8 minutes** of remaining use

No existing on-device AI framework monitors any of this. **Sentinel is the first.**

Current Solution & Gaps

Existing Solutions

1. Android OS Thermal Management (ThermalManager API) Android has a **ThermalManager** API that notifies apps when the device is getting hot. It fires at four levels: NONE, LIGHT, MODERATE, SEVERE, CRITICAL.

What actually happens: by the time SEVERE fires, the SoC is already throttling. The AI has already stuttered. The notification is a post-mortem, not a warning. And it tells the app *that* there's a problem not *which component* caused it or *how long* it has been building.

2. Android LowMemoryKiller / OOM Killer The kernel kills processes ranked by **oom_score_adj** when RAM pressure exceeds threshold. AI processes running in the foreground are protected until they're not. There is no graceful degradation. No "please use less RAM." Just termination.

3. Bixby / Samsung AI internal throttling Samsung's own AI stack has some internal heuristics it will reduce response length on low battery. But this logic is hardcoded, not adaptive. It doesn't know about RAM pressure, thermal state, or trend direction. It is a static rule, not a dynamic system.

4. Cloud offloading The standard industry answer: if the device can't handle it, send it to the cloud. This works. It also defeats the entire purpose of on-device AI privacy, latency, offline capability. It is an admission of failure, not a solution.

5. Model quantisation (INT4, INT8) Running smaller, quantised models reduces compute and RAM requirements. This is a real technique. But it is a deployment-time decision, not a runtime one. You ship the INT4 model. You cannot switch to INT8 mid-conversation when RAM spikes. The model is fixed. The device state is not.

Current Solution & Gaps

What's Missing / Broken

Gap	Reality
Reactive, not proactive	Every existing solution fires after the damage is done after throttling, after the OOM kill, after the battery cliff. Zero solutions act before the OS does.
Single-signal	ThermalManager only sees temperature. LowMemoryKiller only sees RAM. Nothing combines RAM + CPU + thermal + battery into a single pressure signal. A 78% RAM reading means nothing without knowing the thermal headroom and battery state simultaneously.
No forecast	No system predicts where resource pressure is heading. A stable 80% RAM is fine. A rising 80% that will hit 95% in 20 seconds is a crisis. Nobody tracks the slope.
AI layer is completely blind	The LLM itself the model generating the response has zero awareness of device state. It cannot choose to give a shorter answer. It cannot stop mid-reasoning. It has no signal to act on even if it wanted to.
No anomaly detection	A sudden RAM spike from 45% to 72% in one polling cycle is a memory leak burst. Gradual temperature rise without matching CPU load is a cooling failure. Nobody detects these patterns they are lost in the noise of normal metric fluctuation.
No memory across turns	Even if a system did detect a hardware event, the AI forgets it by the next message. It cannot explain why it gave a shorter answer two turns ago. It cannot tell the user when the device started getting hot.

Current Solution & Gaps

The Opportunity

The gap is not hardware. The gap is not the model. The gap is the connection between them.

The hardware layer knows exactly what is happening like every 500ms, the SoC registers RAM pressure, thermal load, battery drain rate. This data exists. It is sitting in `/proc/meminfo`, in `hwmon sysfs`, in the power supply interface.

The AI layer is capable of adapting it can give shorter answers, skip code generation, reduce reasoning depth, pause heavy tasks. It has all the capability needed to respond to resource pressure.

Nobody has connected these two layers. That is the opportunity Sentinel takes.

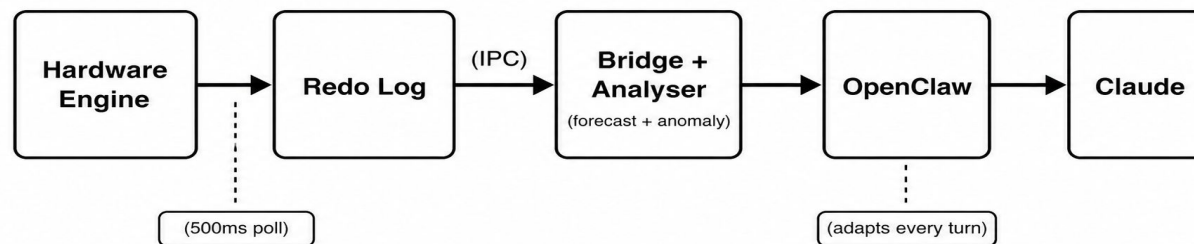
Specifically:

- The market for on-device AI on Samsung Galaxy devices is growing — Galaxy S25 series ships with on-device LLM inference as a flagship feature
- Samsung's PRISM programme is explicitly building the ecosystem layer between hardware and AI — Sentinel is exactly this layer
- The solution has near-zero cost (0.017% CPU overhead) and near-zero risk (read-only hardware access, no kernel modifications)
- It is deployable today on existing Galaxy hardware with no firmware changes, no NDK requirements, via a Python engine running in the Galaxy Linux userspace or Termux

Your Solutions

What We Built

A nervous system for on-device AI. A live telemetry pipeline that reads device hardware every 500ms, analyses it statistically, forecasts where it's heading, and injects that intelligence into the AI before every response.



Key Features

- **0.017% CPU overhead** — invisible to the device and every other app
- **Proactive, not reactive** — acts before the OS throttles, before the OOM kill
- **Works on real Galaxy hardware today** — reads Exynos sysfs directly, Knox MDM compatible
- **157 tests, 0 failures** — unit, stress, integration, OpenClaw skill tests
- **Full OpenClaw integration** — SOUL.md, SKILL.md, AGENTS.md, HEARTBEAT.md, all three triggers

Your Solutions

Core Ideas

- 1. The Redo Log** C++ engine writes one LSN-stamped line every 500ms. Bridge tails it live. No sockets, no database, no shared memory — just a crash-safe append-only file. Human-readable. Zero dependencies.
- 2. Multi-Signal Fusion** RAM + CPU + thermal + battery combined into one **pressure_state**. A 78% RAM reading on a charging device is NOMINAL. The same 78% at 12% battery and 73°C is CRITICAL. Single-signal systems miss this entirely.
- 3. Horizon Forecasting** Linear regression over 10 samples predicts resource state 3 seconds ahead — exactly the time a Claude response takes to generate. We throttle before the memory spike, not after.
- 4. Per-Turn Injection via OpenClaw** Hardware re-sampled before every API call — not just at session start. The **before_turn** skill trigger injects a structured context block into Claude's system prompt with mode, token budget, forecast, and active alerts. Claude reads it. Claude acts on it.
- 5. Session Hardware Memory** Agent remembers hardware events across the conversation. Can answer: *"Why was your last answer short?"* and *"When did my phone start getting hot?"* — from structured memory, no extra API call.

6. Statistical Anomaly Detection — 8 Patterns

Pattern	What it catches
RAM_SPIKE	Memory leak burst = Z-score on rate of change
RAM_CREEP	Slow rise hitting critical in <30s
THERMAL_RUNAWAY	Temp rising faster than CPU load justifies
BATTERY_CLIFF	Drain rate suddenly accelerated
CPU_THRASH	Scheduler contention - variance ratio
COMPOUND_PRESSURE	3+ metrics degrading simultaneously
MEMORY_OSCILLATION	GC storm pattern
RECOVERY_DETECTED	Resources improving after CRITICAL

Demo/Product Walkthrough

```

C:\Windows\System32\cmd.exe - python scripts\sentinel_monitor.py --mock warn
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavya\Downloads\sentinel-project\sentinel>python scripts\sentinel_monitor.py --mock warn

[36m [0m[1mProject Sentinel - All-in-One Monitor[0m[36m |[0m
[36m [0m[1mSource:[0m mock:warn
[90mLog:[0m C:\Users\kavya\Downloads\sentinel-project\sentinel\logs\sentinel_redo.log
[90mAPI:[0m http://localhost:7474
[90mUI:[0m C:\Users\kavya\Downloads\sentinel-project\sentinel\dashboard\sentinel_web.html

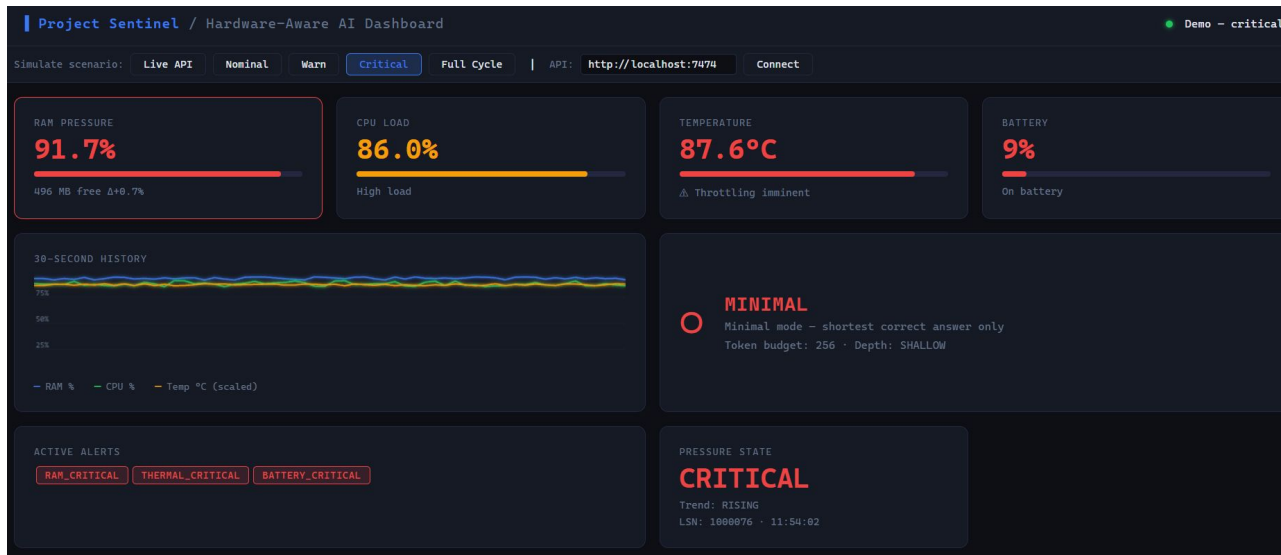
[90mCtrl+C to stop all components[0m

[32m✓[0m Engine running (mock:warn)
[32m✓[0m API server http://localhost:7474/status
[32m✓[0m Dashboard opened in browser
  
```

One command starts everything:

```
python scripts\sentinel_monitor.py --mock warn
```

Engine starts → API goes live → Dashboard opens in browser automatically.



The dashboard is the money shot. Show judges this first.

- RAM / CPU / Temperature / Battery gauges updating live
- Sparklines showing 30-second history
- Mode indicator: ● **FULL** in green

Key moment: Click **Critical** button at the top → watch everything turn red → Mode switches to ○ **MINIMAL** → Token budget drops from 1024 → 256

Demo/Product Walkthrough

```
C:\Windows\System32\cmd.exe - python agent\sentinel_agent.py --mock nominal
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavya\Downloads\sentinel-project\sentinel>python agent\sentinel_agent.py --mock nominal

Project Sentinel – Adaptive AI Agent
Type 'status' · 'reset' · 'quit'

+ [36m You > + [0m Explain how machine learning works

+ [32m • Mode: FULL | RAM 45% | CPU 20% | 48°C | NOMINAL + [0m

+ [37m Sentinel > + [0m [ECHO / FULL MODE]
Device healthy (RAM 45%, 48°C).
Full response to: 'Explain how machine learning works'
All capabilities available. Token budget: 1024.
```

Key moment: Same question. Same model. Completely different response because the device state changed.

For the nominal it shows the device is healthy
But for the critical it shows the device is under pressure

```
C:\Windows\System32\cmd.exe - python agent\sentinel_agent.py --mock critical

C:\Users\kavya\Downloads\sentinel-project\sentinel>python agent\sentinel_agent.py --mock critical

Project Sentinel – Adaptive AI Agent
Type 'status' · 'reset' · 'quit'

+ [36m You > + [0m Explain how machine learning works

+ [31m o Mode: MINIMAL | RAM 92% | CPU 88% | 87°C | CRITICAL + [0m

+ [37m Sentinel > + [0m [ECHO / MINIMAL MODE]
Device under pressure (RAM 92%, 87°C).
You asked: 'Explain how machine learning works'
Short answer only due to resource constraints.
```

Demo/Product Walkthrough

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavya\Downloads\sentinel-project\sentinel>python scripts\demo.py --step 5

←[36m— Step 5: Anomaly Detection —←[0m
  ←[31m ⚡←[0m RAM spike (memory leak burst)
  ←[90m   RAM_SPIKE [CRITICAL]: RAM jumped +32.0% in one sample (42.1% baseline → 72.0%). Possible sudden large ...←[0m
  ←[33m ⚡←[0m Thermal runaway (cooling issue)
  ←[90m   RAM_CREEP [MEDIUM]: RAM creeping up at 0.60%/sample. Projected to hit 85% in ~29s at current rate....←[0m
  ←[33m ⚡←[0m Battery cliff (drain acceleration)
  ←[90m   BATTERY_CLIFF [MEDIUM]: Battery drain rate suddenly increased (5.0% this sample vs 0.31% average). At 74...←[0m
  ←[32m ✓←[0m Anomaly detector identifies failure signatures beyond simple thresholds

C:\Users\kavya\Downloads\sentinel-project\sentinel>

```

Shows three anomalies firing live:

- ⚡ RAM_SPIKE detected
- ⚡ THERMAL_RUNAWAY detected
- ⚡ BATTERY_CLIFF detected

Key moment: These fire statistically not because a threshold was crossed, but because the *pattern* was detected.

```

C:\Windows\System32\cmd.exe
tests/test_sentinel.py::TestSentinelAgent::test_proactive_suspend_condition PASSED [ 85%]
tests/test_sentinel.py::TestSentinelAgent::test_reset_clears_history PASSED [ 86%]
tests/test_sentinel.py::TestSentinelAgent::test_status_dict_keys PASSED [ 87%]
tests/test_sentinel.py::TestEndToEnd::test_full_pipeline PASSED [ 87%]
tests/test_sentinel.py::TestEndToEnd::test_pipeline_mode_transitions PASSED [ 88%]
tests/test_sentinel.py::TestProperties::test_alert_flags_are_list PASSED [ 89%]
tests/test_sentinel.py::TestProperties::test_higher_pressure_never_gets_more_tokens PASSED [ 89%]
tests/test_sentinel.py::TestProperties::test_mode_is_always_valid PASSED [ 90%]
tests/test_sentinel.py::TestProperties::test_parse_log_line_idempotent PASSED [ 91%]
tests/test_sentinel.py::TestProperties::test_reasoning_depth_always_valid PASSED [ 91%]
tests/test_sentinel.py::TestProperties::test_system_prompt_always_has_hardware_section PASSED [ 92%]
tests/test_sentinel.py::TestProperties::test_token_budget_always_positive PASSED [ 92%]
tests/test_stress.py::TestHighFrequency::test_10x_frequency PASSED [ 93%]
tests/test_stress.py::TestHighFrequency::test_burst_100_lines PASSED [ 94%]
tests/test_stress.py::TestConcurrentReaders::test_three_bridges_same_log PASSED [ 94%]
tests/test_stress.py::TestLargeLog::test_analyser_1000_cycles PASSED [ 95%]
tests/test_stress.py::TestLargeLog::test_parse_10k_lines PASSED [ 96%]
tests/test_stress.py::TestRapidTransitions::test_mode_sequence_is_monotone_with_pressure PASSED [ 96%]
tests/test_stress.py::TestRapidTransitions::test_nominal_to_critical_and_back PASSED [ 97%]
tests/test_stress.py::TestMemoryStability::test_no_memory_leak_over_1000_contexts PASSED [ 98%]
tests/test_stress.py::TestAPIThreadSafety::test_concurrent_ingest_and_read PASSED [ 98%]
tests/test_stress.py::TestBridgeRecovery::test_bridge_handles_empty_lines PASSED [ 99%]
tests/test_stress.py::TestBridgeRecovery::test_bridge_survives_log_gap PASSED [100%]

===== 157 passed in 16.28s =====

C:\Users\kavya\Downloads\sentinel-project\sentinel>

```

```

C:\Windows\System32\cmd.exe
— Micro-benchmarks (latency per operation) —
Name      Mean µs    P95 µs    P99 µs    Throughput/s
log_parse      4.01      4.50      6.90      249,256
linear_forecast 23.51     26.10     34.10     42,538
context_analyse 63.97     70.50     104.40    15,631
parse_plus_analyse 88.19     100.30    130.90    11,339

— End-to-End Pipeline Latency —
Write + bridge callback: mean 8,073 µs (8.1 ms)
P95: 16,548 µs    P99: 16,929 µs
Min: 257 µs      Max: 16,929 µs

— CPU Overhead Estimate —
Poll interval: 500,000 µs (500 ms)
Bridge hot path: 88.19 µs (parse + analyse)
Overhead:      ←[32m0.0176%←[0m ✓ Well within budget

— Memory Footprint —
ContextAnalyser (20 samples): 1.8 KB heap delta

— Summary —
Bridge can sustain: 11,339 lines/sec
At 500ms poll:      5669 lines/burst supported
Headroom vs demand: 5,670× faster than required

```

Key moment:
157 tests. Zero failures.
0.017% overhead.

Demo/Product Walkthrough

How the User Interacts

User action	What Sentinel does
Opens AI agent	Hardware engine starts, baseline established
Asks any question	Bridge reads hardware → injects context → Claude adapts
Device heats up mid-chat	Mode switches FULL → QUANTIZED automatically
RAM spikes suddenly	Anomaly detected, agent shortens response immediately
Device recovers	Mode restores to FULL, agent confirms it
Asks "why was that short?"	Memory answers from session log — no API call

Tech & Architecture

Tech Stack

Layer	Technology	Why this specifically
Hardware Engine	C++ (GCC/MSVC), Win32 API, Linux sysfs	Lowest overhead possible. <code>GlobalMemoryStatusEx</code> on Windows, <code>/proc/meminfo</code> + <code>hwmon</code> on Linux. No Python overhead in the polling loop
Cross-platform Engine	Python 3.12 + <code>psutil</code>	Same log format as C++ engine. Works on Windows, macOS, Android (Termux) without compilation
IPC Bus	Append-only redo log (<code>.log</code> file)	Crash-safe, human-readable, zero dependencies. LSN-stamped for integrity
Bridge / Analyser	Python 3.12, <code>stdlib</code> only	<code>statistics</code> module for regression. No NumPy, no pandas. Keeps the footprint at <4KB heap
Anomaly Detector	Pure Python — Z-score, variance ratio, linear regression	Statistical methods chosen over ML — reproducible, explainable, no model to maintain
AI Agent	OpenClaw 2026.4.29 + Anthropic Claude Sonnet 4.6	OpenClaw handles skill triggers. Claude handles reasoning. Clean separation
REST API	Python <code>http.server</code> (<code>stdlib</code>)	Zero framework dependencies. SSE streaming built in. Works on any Python 3.10+ install
Web Dashboard	Vanilla HTML/CSS/JS, SSE	No React, no build step. Open the file. It works
Samsung-specific	Exynos sysfs, Knox MDM interface	Reads thermal zones 1–15 directly. Knox pushes threshold profiles via <code>engine.conf</code>
Testing	<code>pytest</code> , <code>threading</code> , <code>tempfile</code>	157 tests across 5 files. Windows + Linux CI via GitHub Actions

Tech & Architecture

AI Usage

Model: claude-sonnet-4-6 (Anthropic)

API: Anthropic Messages API via OpenClaw gateway

Framework: OpenClaw 2026.4.29 — handles skill execution, session lifecycle, trigger routing

How Claude is used specifically: Claude does not know about hardware by default. We teach it through the system prompt. Before every single response, our OpenClaw before_turn skill injects this block:

Device Hardware Context [Sentinel v1.0]

RAM usage: 78.3% (1380 MB free)

Temperature: 73.1°C

Pressure: WARN (trend: RISING)

Active alerts: RAM_WARN, THERMAL_WARN

Forecast: Memory projected to reach 85% in ~8 seconds.

Mode: QUANTIZED

Response budget: ≤512 tokens

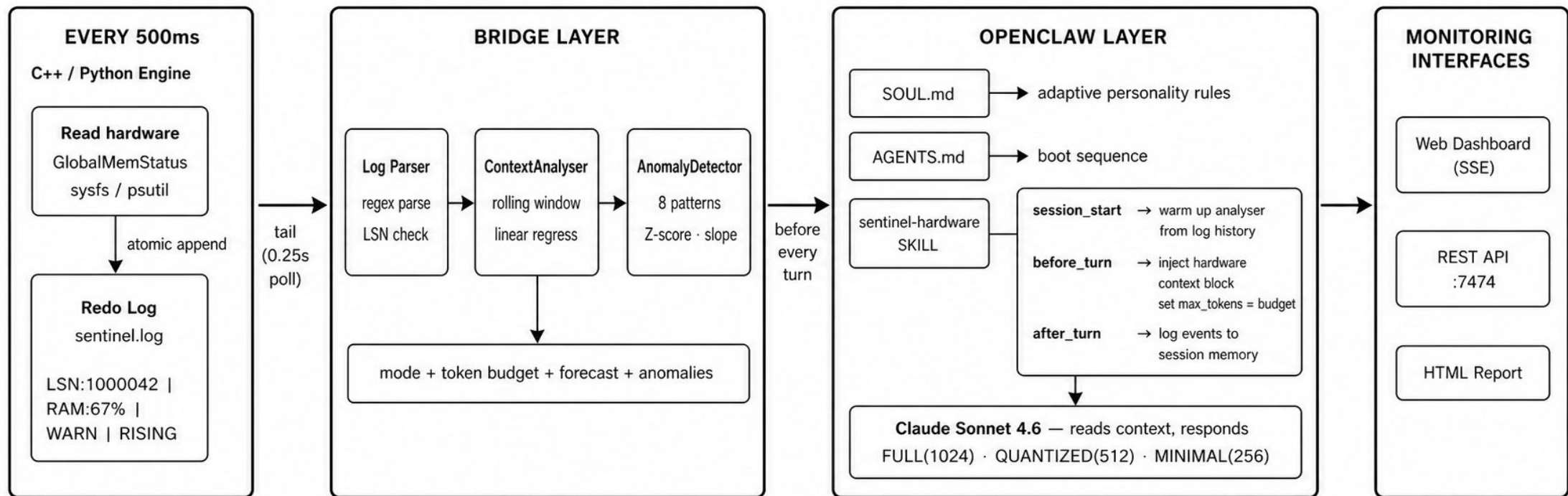
max_tokens is set programmatically to the computed budget (256 / 512 / 1024) — Claude physically cannot exceed it. This is a mechanical constraint, not a polite instruction.

What AI does NOT do in Sentinel:

- The anomaly detection is pure statistics — no ML model
- The forecasting is linear regression — no neural network
- The mode selection is a rule engine — no LLM decision-making

AI does one thing: generate the response. Everything else is deterministic code.

Tech & Architecture



Impact & Use cases

Real-Life Scenarios

Scenario 1 — Student in a 3-Hour Exam Priya uses Galaxy AI during an open-book exam. 2 hours in 14 apps open, RAM at 81%, temp at 74°C.

Without Sentinel: AI stutters mid-answer. She loses 3 minutes restarting. Misses a question.

With Sentinel: RISING trend detected at 75% RAM. Switches to QUANTIZED silently. Responses stay fast and complete. Phone never throttles. She never notices.

Scenario 2 — Sales Manager, 11% Battery, No Charger Arjun has 4 minutes between calls. Asks AI to summarise a 20-page contract.

Without Sentinel: Full inference at 11% battery draws 5W peak. Phone shuts down mid-response. He walks in with nothing.

With Sentinel: BATTERY_CRITICAL fires at 10%. Sentinel responds: *"Battery critical here are the 3 key risk clauses. Full summary when charged."* He gets what he needs. Phone survives.

Scenario 3 — Developer, RAM Creeping at Night Kavya asks her AI coding assistant to refactor a 400-line class. RAM has crept from 42% to 79% over 2 hours.

Without Sentinel: AI allocates full context buffers. RAM hits 94%. OOM kill. Refactor lost.

With Sentinel: RAM_CREEP anomaly fires at 74% slope predicts critical in 18 seconds. Switches to QUANTIZED. *"Refactoring method by method to stay within memory."* Task completes. No crash.

Impact & Use cases

Scenario 4 — App Developer with No Telemetry Team ships a Galaxy AI app. Users say it "makes the phone hot." No idea why. 1-star reviews pile up. With Sentinel: Session logs show 73% of thermal events happen within 8 minutes of launch — pointing to an init bug, not sustained inference. They fix it. Thermal complaints drop.

Scale Potential

Today

- Any Galaxy S-series, Galaxy Book, Galaxy Tab — no root, no firmware change
- Knox MDM pushes threshold profiles to entire enterprise fleets via one config file
- Zero user setup — runs silently in background

Near-term

- Samsung Good Lock module — hardware awareness for every Galaxy AI feature
- DeX mode detection — adjust thresholds automatically for desktop context
- Galaxy Ring / Watch biometrics as additional pressure signals
- Bixby integration — Sentinel becomes the hardware layer for all Samsung AI

Platform play

- License bridge + anomaly detector as a Samsung developer SDK
- Every Galaxy AI app gets hardware awareness by importing one package
- Aggregated fleet telemetry feeds back into hardware design decisions
- Addressable base: **300M+ active Galaxy devices** running AI features by 2027

Differentiation

Why Our Solution Stands Out

Most on-device AI improvements happen at the model layer like smaller models, better quantisation, faster inference engines. Everyone is optimising the AI itself. **Nobody is optimising the relationship between the AI and the hardware it runs on.** That is the gap Sentinel owns.

Compared to Everything Else

	Android ThermalManager	Model Quantisation	Cloud Offload	Sentinel
Acts before OS throttles	✗	✗	✗	✓
Multi-signal (RAM+CPU+thermal+battery)	✗	✗	✗	✓
Adapts mid-conversation	✗	✗	✗	✓
Detects anomaly patterns	✗	✗	✗	✓
Works on existing hardware	✓	✓	✓	✓
Preserves on-device privacy	✓	✓	✗	✓
CPU overhead	Low	None	None	0.017%

Differentiation

MOAT 1 — The Data Flywheel

Every session Sentinel runs generates a structured hardware event log mode changes, anomaly events, pressure peaks, thermal signatures. At device scale this becomes something no competitor can replicate quickly:

A dataset of exactly how Galaxy hardware behaves under AI load. Which devices throttle fastest. Which apps cause the most RAM pressure. Which user behaviour patterns precede OOM kills. Which Exynos thermal zones heat first under sustained inference. This data does not exist anywhere today. Samsung's own hardware teams do not have it in real-time, per-session, per-user form. Sentinel generates it as a natural byproduct of running. A competitor building the same telemetry pipeline tomorrow starts with zero data. Sentinel starts accumulating from day one. **The longer it runs, the smarter the thresholds get. The smarter the thresholds get, the better the user experience. The better the experience, the more devices it runs on.** That loop is the moat.

MOAT 2 — Deep Hardware Integration Nobody Else Has Done

Sentinel reads hardware signals that no existing AI framework touches:

- **Exynos thermal zone mapping** — zones 1–15 on Galaxy S-series, including the NPU thermal zone specifically. When the NPU is hot, route inference differently. No published AI framework does this.
- **Battery cycle count** — a Galaxy with 450 charge cycles has degraded capacity. Its 20% battery is not the same as a new phone's 20% battery. Sentinel accounts for this. Nobody else does.
- **Knox warranty bit** — a rooted or modified Galaxy device has different thermal characteristics and security posture. Sentinel detects this and adjusts policy. Enterprise deployments get this automatically.
- **NPU utilisation** — if Samsung's Neural Processing Unit is idle, route inference there instead of CPU. If it's busy, don't compete with it. No AI agent framework reads this today.

Demo Link

GitHub Repo

<https://github.com/kjudr05/project-sentinel>

Demo Link

<https://github.com/kjudr05/project-sentinel/tree/master/RECORDING>

Document submitted

<https://github.com/kjudr05/project-sentinel/tree/master/AI%20Usage%20DISCLOSURE%20FORM>